endava

.Net - Backend Engineering

Nugets and Tools.

Agenda

What we will cover today in five progressive sections.

01 **Fundamentals**
What NuGet is and why package management matters

02 **Search & Install**
Finding, evaluating, and adding packages to your projects

03 **Versioning & Dependencies**
Semantic versioning, version ranges, and conflict resolution

04 **Custom Packages**
When and how to create, pack, and publish your own packages

05 **Complementary Tools**
dotnet tools, analyzers, CI/CD, and maintenance utilities

01 Fundamentals

What Is NuGet?

The official package manager for .NET — how libraries are shared, versioned, and consumed across the ecosystem.

- **NuGet** is the standard package manager for .NET, similar to npm (Node) or pip (Python).
- Packages are distributed as **.nupkg**.
- **nuget.org** is the public registry with 300k+ packages
- **Integrated** into Visual Studio, dotnet CLI, and CI/CD pipelines by default.
- **The backbone** of dependency management.

Why Package Management Matters

Consistent dependency handling reduces integration bugs, speeds up onboarding, and keeps builds reproducible.

Reproducibility

Lock files and **version pinning** ensure every developer and CI server builds the same code.

Velocity

Reuse proven libraries instead of reinventing solutions. Teams ship features faster.

Security

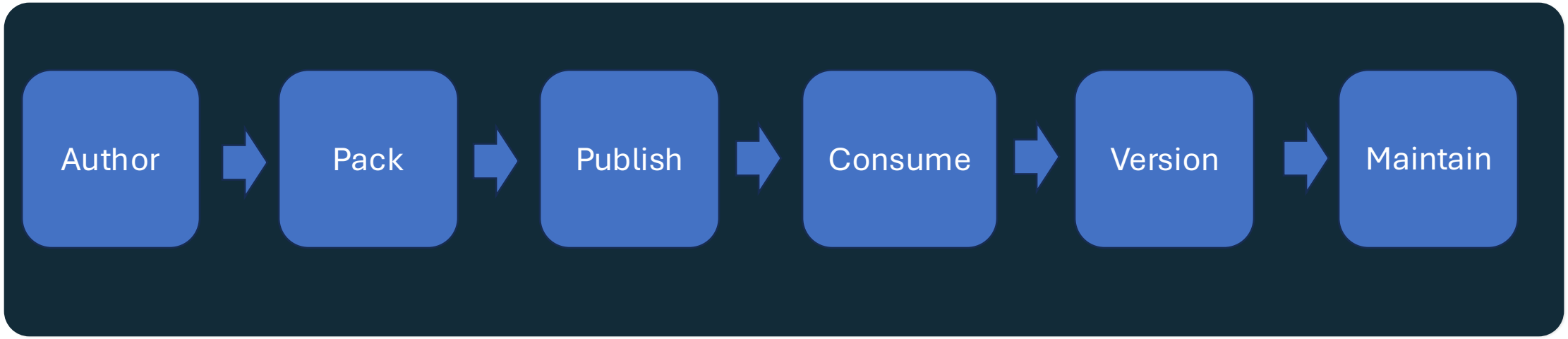
Centralized feeds make it possible to audit, scan, and enforce policies on every dependency.

Maintainability

Clear version contracts make upgrades predictable and reduce the blast radius of breaking changes.

The NuGet Package Lifecycle

From authoring to consumption — four stages every backend engineer should understand.



02 Search & Install

Searching for the Right Package

Finding reliable, well-maintained packages is the first step to safe dependency management.

- **nuget.org** — the primary public registry. Filter by downloads, recent updates, and verified authors.
- **Visual Studio:** Tools → NuGet Package Manager → Browse. See metadata, dependencies, and versions inline.
- **CLI**: dotnet package search <term> (available in .NET 8+) for quick terminal lookups.
- **Evaluate before installing:** check the license, last publish date, open issues, and transitive dependencies.
- **Prefer packages by Microsoft**, well-known OSS authors, or packages with the "prefix reserved" badge.

Installing Packages via CLI

The dotnet CLI is the fastest way to add, update, or remove packages from any .NET project.

dotnet CLI commands

```
# Install a package from CLI
dotnet add package Newtonsoft.Json

# Install a specific version
dotnet add package Serilog --version 3.1.1

# Install from a private source
dotnet add package MyCompany.Shared \
  --source
https://pkgs.mycompany.com/nuget/v3/index.json
```

- dotnet add package is the primary command — it modifies .csproj automatically.
- Specify --version to pin a specific release or use floating ranges.
- Use --source to pull from private or internal feeds alongside nuget.org.
- Visual Studio also offers a GUI: right-click project → **Manage NuGet Packages.**

Package References in .csproj

Every installed package becomes a PackageReference entry in your project file — the single source of truth.

.csproj PackageReference syntax

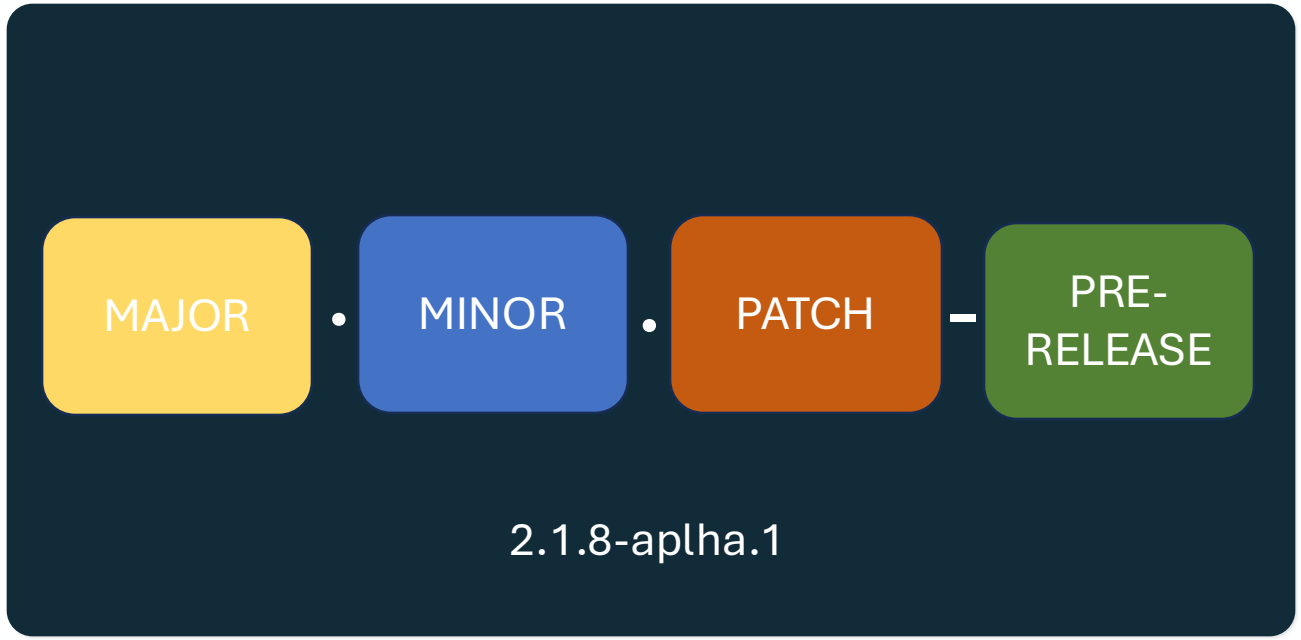
```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Newtonsoft.Json"
                      Version="13.0.3" />
    <PackageReference Include="Serilog"
                      Version="3.1.*" />
    <PackageReference Include="MediatR"
                      Version="[12.0,13.0)" />
  </ItemGroup>
</Project>
```

- PackageReference is the modern format (replaces packages.config).
- Version can be exact (13.0.3), floating (3.1.*), or a range ([12.0,13.0)).
- NuGet resolves the best matching version at restore time.
- Transitive dependencies are resolved automatically — you only list direct references.

03 Versioning & Dependencies

Semantic Versioning (SemVer)

The versioning contract that every NuGet package should follow — and every consumer should understand.



- **MAJOR** — breaking API changes. Consumers must review and adapt code.
- **MINOR** — new features added, fully backward compatible.
- **PATCH** — bug fixes only, no new features, no breaking changes.
- **Pre-release** tags (e.g., 2.0.0-beta.1) signal unstable or preview builds.
- Always read the changelog before upgrading a major version.

Version Ranges and Pinning Strategies

NuGet supports flexible version expressions — choose the right strategy for stability vs. freshness.

Version range examples in .csproj

```
<!-- Exact version -->
<PackageReference Include="PkgA" Version="2.1.0" />

<!-- Minimum inclusive -->
<PackageReference Include="PkgB" Version="2.1.*" />

<!-- Range: >= 1.0 and < 2.0 -->
<PackageReference Include="PkgC" Version="[1.0,2.0)" />

<!-- Floating: latest patch of 3.x -->
<PackageReference Include="PkgD" Version="3.*" />
```

- **Exact pinning** — maximum reproducibility, manual upgrades only.
- **Minimum (2.1.*)** — auto-adopt patches/minors on each restore.
- **Range ([1.0,2.0))** — allow updates within a major version, block breaking changes.
- **Floating (3.*)** — auto-adopt patches/minors on each restore.

Dependency Conflicts and Resolution

When two packages require different versions of the same transitive dependency, NuGet must decide which one wins.

- NuGet uses a nearest-wins rule: the version closest to your project in the dependency graph is selected.
- Conflicts surface as NU1605 or NU1608 warnings at restore time — never ignore them.
- Use dotnet list package --include-transitive to visualize the full dependency tree.
- Override a transitive version by adding an explicit PackageReference in your project.
- Central Package Management (Directory.Packages.props) prevents version drift across a multi-project solution.

Best Practices for Stable Dependencies

Practical guidelines that keep your dependency graph healthy and your builds predictable.

Pin & Audit

- Pin critical packages to exact versions.
- Run dotnet list package --outdated regularly.
- Enable NuGet Audit in your .csproj for CVE checks.
- Use lock files (packages.lock.json) for full reproducibility.

Centralize

- Use Directory.Packages.props for multi-project solutions.
- Define all versions in one place.
- Prevent conflicting version declarations across projects.
- Enforce consistency through CI validation.

Automate

- Enable Dependabot or Renovate for PR-based updates.
- Run automated tests on every dependency bump.
- Set merge policies that require green builds.
- Schedule monthly dependency review sessions.

Central Package Management

A single Directory.Packages.props file governs all package versions across every project in the solution.

Directory.Packages.props

```
<!-- Directory.Packages.props (repo root) -->
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true
    </ManagePackageVersionsCentrally>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Serilog"
                    Version="3.1.1" />
    <PackageVersion Include="MediatR"
                    Version="12.2.0" />
  </ItemGroup>
</Project>
```

- Place at the repository root — NuGet discovers it automatically.
- Individual .csproj files reference packages without specifying versions.
- Upgrades happen in one PR, one file, tested once — no version drift.
- Ideal for mono-repos and microservice solutions with shared libraries.

04 Custom Packages

When to Create a Custom NuGet Package

Not everything should be a package — but shared code that crosses project boundaries often should be.

- Shared utilities consumed by 2+ services or solutions (e.g., logging, auth, DTOs).
- Internal SDKs that wrap third-party APIs with company-specific conventions.
- Cross-team contracts: shared models, interfaces, or middleware components.
- Do NOT package code that changes with every sprint — it creates coupling, not reuse.
- A good custom package has a clear API surface, its own tests, and a versioned release process.

Creating, Packing, and Publishing a Package

From .csproj metadata to a published .nupkg — the full authoring workflow in three commands.

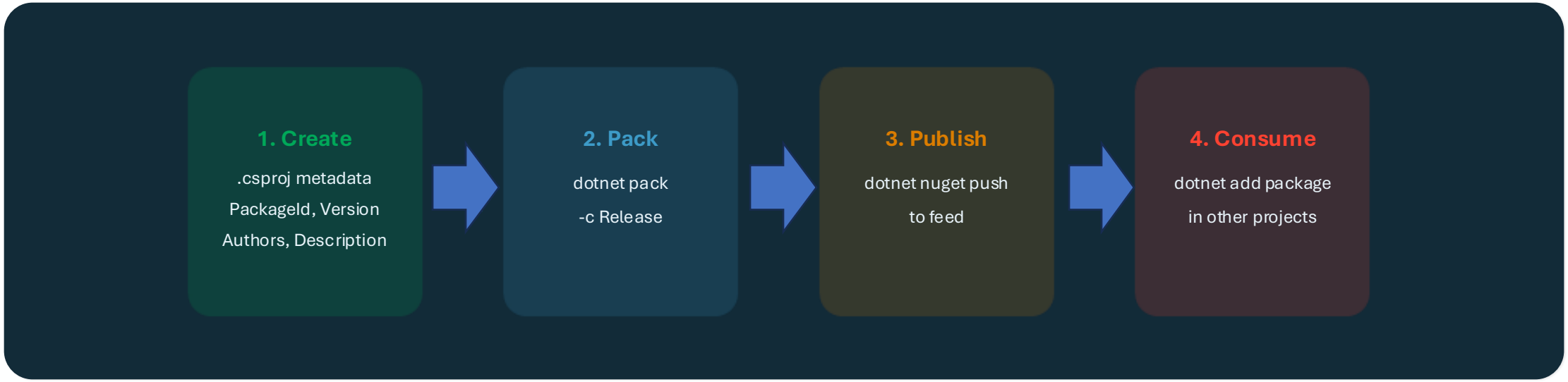
Package authoring workflow

```
# Add metadata to your .csproj
<PropertyGroup>
  <PackageId>MyCompany.Utils</PackageId>
  <Version>1.0.0</Version>
  <Authors>Backend Team</Authors>
  <Description>Shared utilities</Description>
</PropertyGroup>

# Build the package
dotnet pack -c Release

# Push to a feed
dotnet nuget push bin/Release/*.nupkg \
  --source https://nuget.mycompany.com/v3 \
  --api-key <YOUR_KEY>
```

- **Step 1**: Add metadata to your .csproj (PackageId, Version, Authors, Description).
- **Step 2:** Run dotnet pack -c Release to generate the .nupkg file.
- **Step 3:** Push to your feed with dotnet nuget push.
- **Automate this in CI** — every merge to main publishes a new pre-release version.

Package Authoring Pipeline

A four-stage pipeline from source code to reusable package consumed by other projects.



Publish*

- nuget.org for public,
- Azure Artifacts or GitHub Packages for private

05 Complementary Tools

Complementary Tools in the .NET Ecosystem

NuGet does not work alone — these tools cover quality, testing, CI/CD, and maintenance.

.NET Ecosystem Tools

Package Mgmt

NuGet CLI · dotnet CLI
Paket · NuGet.org

Code Quality

Roslyn Analyzers · SonarQube
StyleCop · EditorConfig

Testing

xUnit · NUnit · Moq
FluentAssertions · Coverlet

CI/CD

GitHub Actions · Azure Pipelines
dotnet build/test/publish

Maintenance

Dependabot · dotnet outdated
NuGet Audit · dotnet-format

Performance

BenchmarkDotNet
dotnet-counters · dotnet-trace

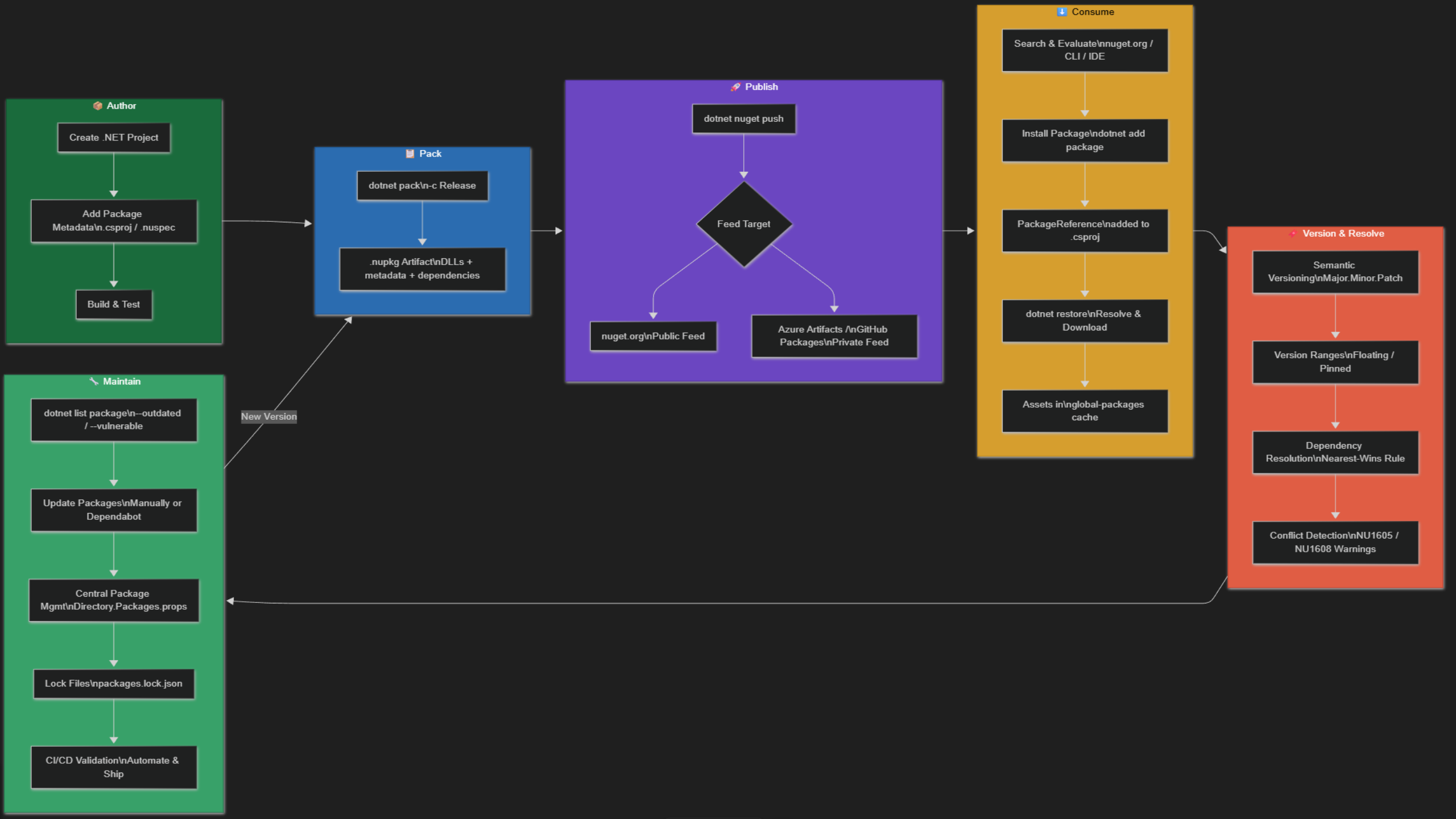
dotnet Tools — CLI Extensions

Global and local dotnet tools extend your command line with project-specific or team-wide utilities.

```
# Install a global tool
dotnet tool install -g dotnet-outdated-tool

# Check which packages need updates
dotnet outdated

# Install a local tool manifest
dotnet new tool-manifest
dotnet tool install dotnet-format

# Run the local tool
dotnet tool run dotnet-format
```

- **Global tools** install once and are available everywhere (e.g., dotnet-outdated, dotnet-ef).
- **Local tools** live in a .config/dotnet-tools.json manifest — versioned with the repo.

06 The Full Picture


Author
Create .NET Project
Add Package Metadata\n.csproj / .nuspec
Build & Test
Pack
dotnet pack\n-c Release
.nupkg Artifact\nDLLs + metadata + dependencies
Publish
dotnet nuget push
Feed Target
nuget.org\nPublic Feed
Azure Artifacts /\nGitHub Packages\nPrivate Feed
Consume
Search & Evaluate\nnuget.org / CLI / IDE
Install Package\ndotnet add package
PackageReference\nadded to .csproj
dotnet restore\nResolve & Download
Assets in\nglobal-packages cache
Version & Resolve
Semantic Versioning\nMajor.Minor.Patch
Version Ranges\nFloating / Pinned
Dependency Resolution\nNearest-Wins Rule
Conflict Detection\nNU1605 / NU1608 Warnings
Maintain
dotnet list package\n--outdated / --vulnerable
Update Packages\nManually or Dependabot
Central Package Mgmt\nDirectory.Packages.props
Lock Files\npackages.lock.json
CI/CD Validation\nAutomate & Ship
New Version

Key Takeaways

The essential points to carry into your next .NET project.

- NuGet is the backbone of .NET dependency management — understand its lifecycle end to end.
- Use semantic versioning and version ranges to balance stability with freshness.
- Central Package Management eliminates version drift across multi-project solutions.
- Create custom packages only for genuinely shared code with a stable API surface.
- Complement NuGet with dotnet tools, analyzers, and automated dependency updates.

- **Task: Create a Custom NuGet Package (Apply all as you can)**

Quick checklist

- Are all packages pinned or using deliberate ranges?
- Is Central Package Management enabled?
- Are dependency updates automated via Dependabot or Renovate?
- Are internal shared libraries published as NuGet packages?
- Are dotnet tools defined in a local manifest?

Thank you!